# B2B Coffee Platform – High-Level Design (HLD)

## 1. Purpose

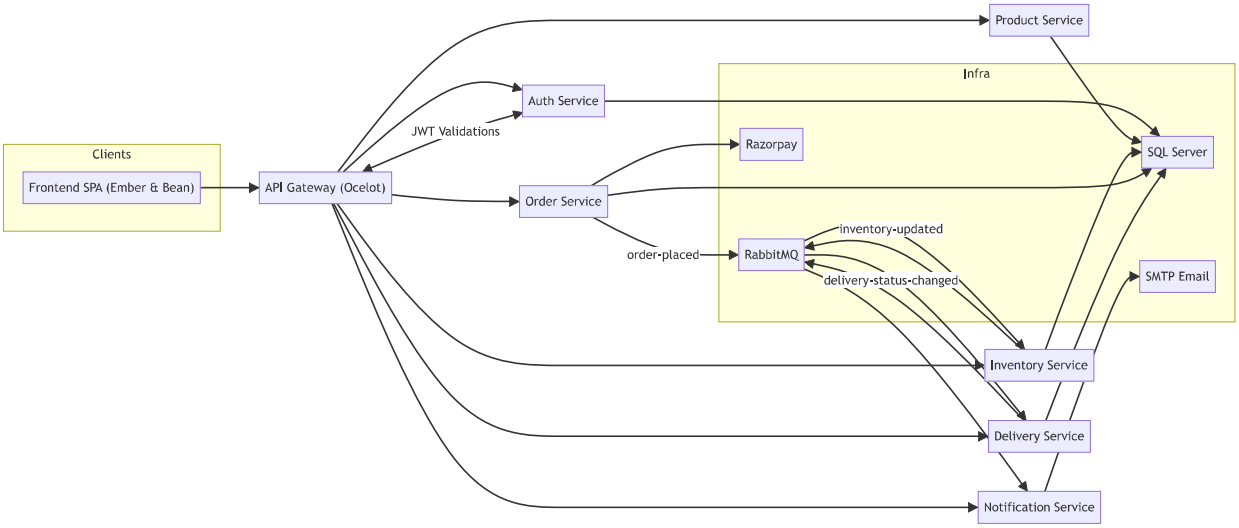
This document summarizes the end-to-end design of the B2B Coffee platform (Ember & Bean) so engineers, testers, and operators share a common understanding before implementation or major changes. It focuses on service boundaries, integration patterns, deployment topology, and key non-functional expectations rather than low-level API or schema details.

## 2. System Context

- **Primary actors:** Clients (B2B coffee buyers, placing and tracking wholesale orders), Administrators (managing catalog, inventory, and order fulfillment), SuperAdmins (platform oversight and admin management), and automated payment/notification providers (Razorpay, SMTP).
- **Entry points:** Angular Frontend SPA (Ember & Bean) communicates through a single Ocelot-based API Gateway.
- **External systems:** Razorpay payment gateway, SMTP provider (for email notifications), and potential analytics/monitoring backends.

## 3. Architecture Overview

The solution follows a microservices style with clear domain ownership per service, asynchronous messaging for order and inventory sagas, and a shared infrastructure baseline (SQL Server, RabbitMQ).




## Design Tenets

1. **Gateway-first security:** The API Gateway validates JWTs, injects identity headers, applies CORS policies, and shields downstream microservices from direct public exposure.
2. **Autonomous services:** Each microservice owns its database context (logical schemas/tables on a shared SQL Server instance) and domain logic, exposing REST APIs that reflect bounded contexts.
3. **Saga orchestration via RabbitMQ:** Order, Inventory, Delivery, and Notification services integrate through domain events (e.g., `order-placed`, `delivery-status-changed`) to provide eventual consistency, process automation, and loose coupling.
4. **Unified Frontend:** The Angular application serves Clients, Admins, and SuperAdmins from a single deployment, utilizing role-based route guards and contextual UIs (the "Living Roast" aesthetic).

# 4. Service Responsibilities

| Service | Domain Ownership | Key Responsibilities | Integrations |
|---|---|---|---|
| **API Gateway (Ocelot)** | Edge security & routing | JWT validation, upstream route mapping, CORS, header enrichment | All backend services |

| | | | |
|---|---|---|---|
| **Auth Service** | Identity & Access | Login/Register, JWT generation, Admin/SuperAdmin management, Client approval/termination | SQL (Auth Db), HTTP (Gateway) |
| **Product Service** | Coffee Catalog | CRUD for coffee products, pricing, features, categorization | SQL (Product Db), REST APIs |
| **Inventory Service** | Stock Management | Tracking stock levels, applying reorder thresholds, deducting stock on orders | SQL (Inventory Db), RabbitMQ |
| **Order Service** | Checkout & Purchasing | Cart management, shipping details collection, Razorpay checkout, initiating order sagas | SQL (Order Db), Razorpay, RabbitMQ |
| **Delivery Service** | Fulfillment Tracking | Delivery status updates (Pending, Dispatched, Delivered), sync with orders | SQL (Delivery Db), RabbitMQ |
| **Notification Service** | Alerts & Email | Sending automated emails for approvals, order placements, and delivery changes | SQL, RabbitMQ, SMTP |
| **Shared** | Contract library | Generic messaging contracts and shared utility models | Consumed by all services |

## 5. Data & Integration Model

- **Datastores:** SQL Server serves as the foundational database engine. Services like `AuthService`, `ProductService`, `OrderService`, `InventoryService`, and `DeliveryService` each manage their respective EF Core DbContexts and migrations.
- **Messaging:** RabbitMQ hosts durable exchanges and queues. Key events include `order-placed` (from Order to Inventory/Delivery/Notification), `inventory-`

`updated`, and `delivery-status-changed` (triggering emails). Consumers use background services to process events decoupled from HTTP requests.

- **HTTP flows:** Direct HTTP calls from Frontend to Gateway, and Gateway routes to specific services. Writes are coordinated via events rather than synchronous HTTP requests.
- **Secrets/config:** Application settings, JWT Secret Keys, Database Connection Strings, and SMTP Credentials managed via `appsettings.json` and environmental variables.

## 6. Critical Flows

1. **Authentication & Authorization**
   - User hits login via API Gateway. `AuthService` validates credentials and issues a JWT containing Roles.
   - Frontend stores the token locally and attaches it as a Bearer token in subsequent requests.
2. **Catalog Browsing & Client Approval**
   - Prospective clients register but require Admin/SuperAdmin approval. Once approved, the Client's state updates, unlocking catalog pricing and ordering capabilities.
3. **Checkout & Order Saga**
   - Client adds products to cart and provides shipping details.
   - `OrderService` initiates Razorpay payment integration for B2B checkout.
   - Upon successful payment verification, `OrderService` persists the order and emits an `order-placed` event.
   - `InventoryService` consumes it to deduct stock; `DeliveryService` initializes a delivery record; `NotificationService` emails the client an order confirmation.
4. **Delivery Orchestration**
   - Admins update the order to "Dispatched" and "Delivered" via the `DeliveryService`.
   - As statuses change, `delivery-status-changed` events are broadcasted, triggering tracking emails.

## 7. Deployment & Operations

- **Containerization:** Planned support for Dockerizing all .NET microservices and the Angular frontend into a unified `docker-compose` topology.
- **Tools & Platforms:** .NET 8, Angular 17+, SQL Server, RabbitMQ.
- **Logging:** Structured Serilog/NLog dumping logs to standard output or basic files, ensuring errors and integration events can be traced locally.

## 8. Non-Functional Considerations

- **UI/UX Aesthetics:** Platform adheres to the "Living Roast" visual language featuring warm parchment backgrounds, amber/espresso color tokens, and glassmorphism.
- **Scalability:** Stateless APIs scale horizontally. Queue-based processing for inventory/emails acts as a shock-absorber during high traffic bursts.
- **Data Consistency:** B2B implementation leans towards eventual consistency; updates propagate asynchronously across microservices without prolonged locking.

## 9. Open Questions / Future Enhancements

1. Should we adopt OpenTelemetry for centralized tracing of cross-service requests and RabbitMQ sagas?
2. Integrating a formalized cache (e.g., Redis) for product catalogs to reduce SQL load during peak browsing.
3. Automating infrastructure provisioning and CI/CD pipelines.